

A computational approach for variants of coloring planar graphs

Arne Claerhout¹, Jorik Jookan¹ and Jan Goedgebeur¹

¹KU Leuven KULAK

arne.claerhout@student.kuleuven.be, jorik.jookan@kuleuven.be, jan.goedgebeur@kuleuven.be

Abstract

In this paper, we study the coloring of planar graphs for different variants of graph colorings. In particular, the proper, odd, conflict-free and unique-maximum vertex colorings are examined. This paper proposes a backtracking algorithm to color planar graphs.

The chromatic numbers of these coloring variants are also studied further. These are the minimum number of colors needed to color any planar graph. Not all chromatic numbers are known, therefore, conjectures exist. This algorithm can be used to disprove or substantiate these conjectures.

The algorithm was constructed by applying optimizations to a naive algorithm. The final implementation is a fully optimized algorithm implemented in C.

The proposed algorithm does not outperform alternatives, but is only around 1.4 times slower, for the proper coloring. The algorithm does have the added benefit of easy expandability for other undiscussed coloring variants. No other programs exist for the other coloring variants, therefore, these can not be compared.

It was also found that the conjectures that haven't been disproven yet are correct for all planar triangulations with 20 or less vertices.

1 Introduction

Graph theory is a field of research in computer science that is older than the computer. Its conception dating back to the 18th century [1].

A graph $G = (V, E)$ is defined as a pair, containing a collection of vertices (V) and edges (E). Each edge is an unordered pair of unique vertices. A *planar graph* is a variant of graphs such that a planar embedding is possible without edges crossing. Planar graphs and graphs will be used interchangeably. The *class of planar graphs* $\mathcal{P}$ is defined as the set of all planar graphs. *Planar triangulations*, a subset of planar graphs, are graphs with the maximum number of edges for some number of vertices, this means the addition of an edge would break the planarity of the graph.

The coloring of graphs is one of the most well-known fields in graph theory. A coloring of a graph is a function assigning each vertex a color $1, \dots, k$. The most famous of these colorings is the proper coloring. This states that two adjacent vertices—vertices sharing an edge—never share the same color.

The chromatic number $\chi(G)$ of a graph G is the minimum number of colors needed to color the graph. A generalization of this is the chromatic number $\chi(\mathcal{P})$ for the class of planar graphs, which is the minimum number of needed colors to color *any* planar graph. For the proper coloring, it was proven that four colors always suffice [2]. This was a special result, as this was the first proof verified using a computer program.

Coloring of graphs has various applications, with uses including: the assignment of frequencies to radio stations, the coloring of maps, scheduling and more.

There exist many variants of graph colorings. One such variant is an *odd coloring*, which was recently introduced by Petruševski and Škrekovski [3] in 2022. A proper coloring of a graph is said to be odd if for each vertex $v \in V$ there exists a color occurring an odd amount of times in the neighborhood of that vertex $N(v)$. The *odd chromatic number* $\chi_{\text{O}}(\mathcal{P})$ is the minimal number of colors needed to color any graph with an odd coloring.

The other variants are conflict-free and unique-maximum colorings.

The *conflict-free coloring*, introduced in 2003 by Even, Lotker, Ron, and Smorodinsky [4], is a coloring such that every vertex has at least one unique color in its neighborhood. This is useful because of applications in wireless networks and more [5].

Now consider a *proper conflict-free coloring* with respect to open- or closed neighborhoods—a conflict-free coloring with an added proper constraint. The conflict-free property is tested on the open- or closed neighborhood of vertices. An open neighborhood of a vertex v is defined as $N(v) = \{w | (v, w) \in E\}$. This does therefore not include the vertex v itself. A closed neighborhood of a vertex v is $N[v] = N(v) \cup \{v\}$, the open neighborhood of v , including the vertex itself. Both colorings have *chromatic numbers* $\chi_{\text{PCFo}}(\mathcal{P})$ and $\chi_{\text{PCFc}}(\mathcal{P})$ respectively, with the o and c signaling open- or closed neighborhoods.

Similarly, an *improper conflict-free coloring* with respect to open- or closed neighborhoods can be defined. These are

the same colorings, without the proper constraint. Again, *chromatic numbers* $\chi_{\text{ICFo}}(\mathcal{P})$ and $\chi_{\text{ICFc}}(\mathcal{P})$ respectively can be defined for both.

Unique-maximum colorings stem from ordered colorings [6], and were studied further in the context of open- and closed neighborhoods by Fabrici, Lužar, Rindošová and Soták [7]. This is a coloring where the maximum color appearing in the neighborhood of a vertex appears exactly once in the neighborhood of that vertex.

Using the same splitting as the conflict-free coloring, four versions of this coloring can be defined. A proper unique-maximum coloring with respect to open- or closed neighborhoods, and an improper unique-maximum coloring with respect to open- or closed neighborhoods. Each respectively having their own *chromatic number* for the class of planar graphs: $\chi_{\text{pUMo}}(\mathcal{P})$, $\chi_{\text{pUMc}}(\mathcal{P})$, $\chi_{\text{iUMo}}(\mathcal{P})$ and $\chi_{\text{iUMc}}(\mathcal{P})$.

1.1 Known bounds

Not all chromatic numbers of these colorings are known. For most, only bounds exist, where some upper bound always suffices to color all graphs, while the lower bound is derived from a graph with the highest *found* chromatic number.

As the unique color for each vertex v in the pCFc-coloring can always be the color of v itself, this coloring is exactly equal to the proper coloring. Therefore, $\chi_{\text{pCFc}}(\mathcal{P})$ is equal to 4, just like the chromatic number of the proper coloring.

The bounds for all colorings are as follows:

- The odd coloring:
 - $5 \leq \chi_{\text{O}}(\mathcal{P}) \leq 8$ (proven by [8] for upper bound, lower bound is C_5)
- the conflict-free colorings:
 - $6 \leq \chi_{\text{pCFo}}(\mathcal{P}) \leq 8$ (proven by [7])
 - $\chi_{\text{pCFc}}(\mathcal{P}) = 4$
 - $4 \leq \chi_{\text{iCFo}}(\mathcal{P}) \leq 5$ (proven by [9] for lower bound and [10] for upper bound)
 - $\chi_{\text{iCFc}}(\mathcal{P}) = 4$ (proven by [11])
- The unique-maximum colorings (all proven by [7]):
 - $6 \leq \chi_{\text{pUMo}}(\mathcal{P}) \leq 10$
 - $6 \leq \chi_{\text{pUMc}}(\mathcal{P}) \leq 8$
 - $5 \leq \chi_{\text{iUMo}}(\mathcal{P}) \leq 10$
 - $4 \leq \chi_{\text{iUMc}}(\mathcal{P}) \leq 6$

1.2 Conjectures

As most of the chromatic numbers are not known. Some authors started formulating conjectures on this problem.

Petruševski and Škrekovski, the originators of the odd coloring, conjectured $\chi_{\text{O}}(\mathcal{P}) = 5$. Other conjectures introduced by Fabrici, Lužar, Rindošová and Soták [7] include:

- $\chi_{\text{pUMo}}(\mathcal{P}) \leq 7$
- $\chi_{\text{pCFo}}(\mathcal{P}) = 6$
- $\chi_{\text{pUMc}}(\mathcal{P}) = 6$
- $\chi_{\text{iCFo}}(\mathcal{P}) = 4$
- $\chi_{\text{iUMo}}(\mathcal{P}) = 5$
- $\chi_{\text{iCFc}}(\mathcal{P}) = 3$
- $\chi_{\text{iUMc}}(\mathcal{P}) = 4$

Since then, the conjecture for the chromatic number of the iCFc-coloring has been disproven.

1.3 Research Problem

In this paper, two research questions are considered:

- **Can we design an algorithm for coloring planar graphs, faster than or just as fast as the existing coloring algorithms, that is also easily expandable to new coloring variants?**
- **In what way is this algorithm useful in helping to prove or disprove conjectures surrounding these coloring variants?**

The focus is mostly laid on the designing of the mentioned algorithm. Further results are also discussed.

The structure of this paper is as follows. First, the algorithm developed is discussed in Section 2 by examining the pseudo-code. Afterwards, the implementation of said algorithm is discussed in Section 3. Here, all notable optimizations applied are studied further. Then, the results of this paper are discussed further in Section 4. Lastly, the threats to validity are discussed in Section 5, ending with the conclusion in Section 6.

2 Algorithm

In this section the developed algorithm is discussed. A backtracking algorithm was used, coloring the vertices one-by-one and pruning where possible. The pseudo-code is found in Algorithms 1, 2, 3 and 4.

Starting with Algorithm 1, this is the ‘overseer’ of the algorithm. Only choosing a maximum color for the graph and letting the recursive algorithm check if the coloring is possible for that maximum. This is also reflected in the pseudo-code.

Before checking the maximum color, the available colors of the vertices are reset to reflect the maximum color (e.g., if the maximum color is 3, the available colors are 1, 2 and 3).

Algorithm 2 is the main checking algorithm. It checks whether the given graph G can be colored given a maximum color max_c . This is done in multiple steps, starting with a base case: if all vertices are colored, a valid coloring is reached, which notifies the previous recursion step.

Otherwise, it loops over all available colors of the given vertex, where permutations are avoided. This means it will check if that available color is one higher or less than the current maximum color in the graph. If so, a lower color should be assigned and this available color should get skipped. This optimization is further discussed in Section 3.1.

Next, the *vertex* is colored with the available color c , and all neighbors of that *vertex* are updated using the `update_neighbors` function. This will essentially update the available colors of all neighbors and those neighbors’ neighbors, and check if the coloring is still possible. If not, the progress made this recursion step is reset, and the next available color is tried. If the coloring is *still* possible, the next recursion step is called with a new *index* and *depth*.

The `update_neighbors` function is therefore the function checking the correctness of the coloring. If the coloring isn’t valid, the graph goes back a recursion step, therefore backtracking. All checks are done while the algorithm is coloring, which is the reason the base case does not check this anymore.

Algorithm 1 search_chromatic_number

Parameters: The coloring K **Output:** Chromatic number

```
1:  $\mathcal{G} \leftarrow$  graphs generated by plantri
2: for  $G \in \mathcal{G}$  do
3:   for  $max\_c \leq 10$  do
4:     set_max_color_for_vertices( $G$ )
5:     if can_color_with_max_c( $G, K, 0, max\_c, 0, 0$ )
6:       return  $max\_c$ 
7:     end if
8:   end for
9: end for
```

After trying all available colors without results, the vertex color is reset and the previous recursion step is notified of failure.

The *depth* variable is used in the `add_colors_back` function. All of the changes in the entire algorithm, meaning all updates for all vertices and all recursion steps, are kept track of. The *depth* is used to log changes and restore them.

Lastly, Algorithm 3 updates neighbors of a given *vertex* for a given color c , the color assigned to *vertex*. Here, the available colors of all uncolored neighbors of *vertex* are updated when the coloring K is proper (e.g., pUMo, *odd*). If this leads to a neighbor having no available colors, the coloring isn't valid and the pruning starts.

Otherwise, the neighborhood of the neighbors are checked. If these are fully colored (because the last vertex that was uncolored, *vertex*, was colored this recursion step), the correctness is checked for coloring K . Again, if this isn't valid, pruning happens and the previous recursion step is notified, otherwise, the algorithm keeps going.

Finally, if the neighborhood of the neighbor still has one vertex left to color, the available colors for this vertex are updated. This is achievable for most coloring variants, except the proper coloring. A concrete example for the odd coloring is worked out in Algorithm 4. Here, the other vertices' colors are used to check compatibility with the available colors of the uncolored vertex u . In this case, the odd constraint is checked, if only one color appears an odd amount of times, u can not have this color. For the other colorings, similar rules are applied to update the vertex u .

It is clear that the algorithm can color in graphs, and find the chromatic number of that graph for a given coloring, since all possible colorings are tried and checked except those that are be pruned early.

3 Implementation

The algorithm was formulated by developing an efficient implementation, and extracting the algorithm from the code. This was done in multiple stages. First, this was developed in Java to test out different data structures and techniques, and to optimize the algorithm. Afterwards, the optimized implementation found in Java, reflecting the algorithm discussed, was then ported over to C. All versions of the implementa-

Algorithm 2 can_color_with_max_c

Parameters: The graph G , coloring K , max. color in graph $max_c_in_g$, max. color max_c , *index* and *depth***Output:** Coloring achieved

```
1: if All vertices colored then
2:   return true
3: end if
4:  $vertex \leftarrow G.vertices[index]$ 
5: for available color  $c$  of  $vertex$  do
6:   if  $\neg is\_UM(K)$  and  $c > max\_c\_in\_g + 1$  then
7:     continue
8:   end if
9:    $vertex.color \leftarrow c$ 
10:   $new\_max\_c\_in\_g \leftarrow \max(max\_c\_in\_g, c + 1)$ 
11:  if update_neighbors( $G, K, vertex, c, depth$ ) then
12:    add_colors_back( $G, depth$ )
13:    continue
14:  end if
15:  if can_color_with_max_c( $G, K, new\_max\_c\_in\_g,$ 
16:     $max\_c, getBestIndex(), depth + 1$ ) then
17:    return true
18:  end if
19:  add_colors_back( $G, depth$ )
20: end for
21:  $vertex.color \leftarrow 0$ 
22: return false
```

tion and other helper scripts are open-source¹.

The implementation uses *plantri*, a fast generator of planar graphs developed by Brinkmann and McKay [12]. *Plantri* is a tool that is able to generate all planar graphs for a given number of vertices. This allows the program to easily generate and color graphs at once when some number of vertices is given.

3.1 Optimizations

Here, the most important optimizations influencing the execution time are explained. For most, the impact on the speed is measured, to justify the usage of these optimizations. All of the comparisons are measured at the relevant stage of development, so that the code could be kept as clean as possible. A slower "optimization" was therefore never implemented again in newer implementations.

For all experiments, a laptop with an intel i7 CPU was used, using windows as the OS and WSL to execute the program. Each experiment was performed three times to counteract noise on the data. All figures used in this section are using these results. Since there are 9 unique colorings to consider, the cumulative needed time for all colorings is used as the performance measure. All data for all coloring variants is also found here¹.

Available colors

The first optimization used, is keeping track of the available colors per vertex. This allows the algorithm to prune early whenever a vertex has no possible colors left. Comparing

¹The GitHub repository containing the algorithm.

Algorithm 3 update_neighbors

Parameters: The graph G , coloring K , vertex v to update, color c of v and $depth$ of algorithm

Output: Coloring impossible

```
1: for  $n \in \text{vertex.neighbors}$  do
2:   if is_proper(coloring  $K$ ) and  $n.\text{color} = 0$  then
3:     if remove_available_c( $G, n, c, \text{depth}$ ) then
4:       return true
5:     end if
6:   end if
7:   if  $n.\text{neighbors}$  is fully colored and
   ¬is_correctly_colored( $K, n$ ) then
8:     return True
9:   else if  $n.\text{neighbors}$  contains only one uncolored vertex  $u$  then
10:    if handle_special_case( $K, u, n.\text{neighbors}, \text{depth}$ ) then
11:      return true
12:    end if
13:   end if
14: end for
15: return false
```

Algorithm 4 handle_special_case_odd

Parameters: The coloring $K = \text{odd}$, vertex to update u , neighbors and $depth$

Output: Coloring impossible

```
1: colors_odd = {}
2: for neighbor  $\in \text{neighbors} \setminus \{u\}$  do
3:   if neighbor.color  $\in \text{colors\_odd}$  then
4:     colors_odd = colors_odd  $\setminus \{\text{neighbor.color}\}$ 
5:   else
6:     colors_odd = colors_odd  $\cup \{\text{neighbor.color}\}$ 
7:   end if
8: end for
9: if  $|\text{colors\_odd}| = 1$  then
10:  return remove_available_c( $G, u, \text{colors\_odd}, \text{depth}$ )
11: end if
12: return false
```

this to a naive algorithm, yields expected results. Without any other optimizations, this is only applicable for proper colorings (e.g., pUMo, *proper*). Therefore, there is no effect on improper colorings.

Permutations

As illustrated in Algorithm 2, permutations of already tried colorings of the graph are prohibited, except for unique-maximum colorings. For non-unique-maximum colorings, swapping colors (e.g., *changing all vertices with color 2 to color 3 and vice versa*) still yields a valid coloring. This dramatically cuts down the search space.

This isn't the case for a unique-maximum coloring because swapping two colors doesn't always yield a valid coloring (e.g., *swapping the maximum color with a color occurring twice in a neighborhood*). Therefore, all possible colorings for a graph should be checked for the unique-maximum col-

Analysis of implementations

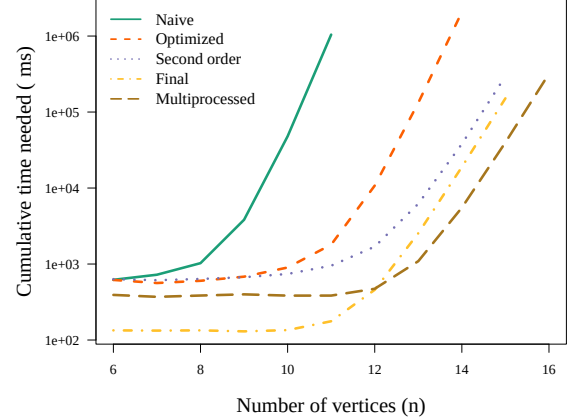


Figure 1: A comparison between different versions of the algorithm, tested on all planar triangulations for a given number of vertices. The naive algorithm is the simplest algorithm described in Algorithm 5. Optimized is the implementation that has implemented the available colors, permutations and bit-sets optimizations. The second order implementation has also implemented the updating of neighbors' neighbors. Lastly, the final and multithreaded implementations are the final C implementation and its multithreaded version using 10 processes respectively. Each implementation performs better than the last.

orings.

Bit-sets

Another optimization used, which is not visible in the pseudo-code of the algorithm, is the use of bit-sets. Instead of each vertex having a list of all neighboring vertices as objects, only the indices of the neighbors are stored, which is done in a binary number. If a vertex v has a neighbor with index i , the neighborhood bit-set of v has a 1 at index i , starting from the least significant bit.

This optimization allows for easier comparison between neighborhoods. Checking if some vertex is present in a neighborhood becomes simple, the lookup time is now constant—as its only a binary operation—instead of a worst-case linear lookup time.

This optimization, combined with the available colors optimization, is implemented in the 'optimized' implementation found in Figure 1. Compared to the naive implementation, a version without these optimizations, the 'optimized' implementation performs better, as the time needed stays low for more n .

Data structures

For the implementation of the algorithm, multiple data structures for monitoring vertices were tested. These include a simple list of vertices, a priority queue and a bit-set implementation. The speed of these data structures are tested against each other.

These structures are all considered because of some strengths each seemingly has. The list of vertices is very simple, and could therefore work well. The priority queue sorts

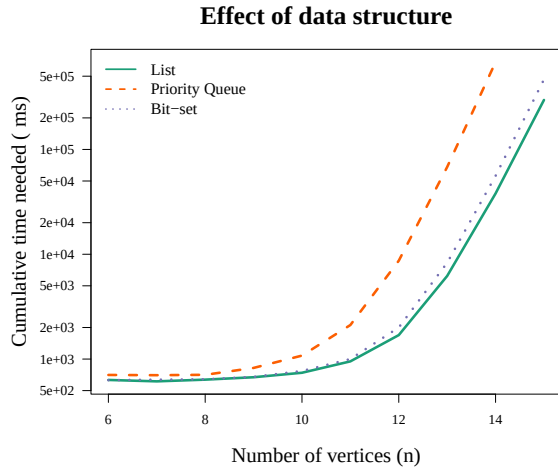


Figure 2: A comparison between the discussed datastructures. The simple list structure performs the best.

the vertices by the amount of available colors (from low to high), and therefore has easy look-up for the best vertex to color in. And lastly, the bit-set data structure, implemented using a list of bit-sets, where each index of the list denotes a number of available colors. The bit-sets then indicate if a vertex either has or doesn't have a certain amount of available colors.

The comparison between datastructures is found in Figure 2. It is clear the priority queue method performs the worst. This is because updating the number of available colors requires a completely new entry in the queue, which has too much overhead. Both the bit-set and standard list implementation perform almost equally well in the figure. Since the scale of the y-axis in the figure is logarithmic, the difference is still substantial. Therefore, going forward, the simple list datastructure is used.

Index

In total, three different `get_best_index`-techniques were tested out. These techniques ranged from simply finding the vertex with the least amount of available colors, to finding the vertex with the highest degree. Some methods were adapted from results found by Aslan and Baykan [13]. Of these 5, the best-performing and simplest methods are compared.

These include the two previously-mentioned methods, respectively called Standard and Welsh Powell (WP), are compared against Standard_WP: a combination of the two techniques, using the standard technique with a degree tiebreaker.

Other methods mentioned by Aslan and Baykan, such as DSATUR and First Fit either had overhead in registering extra data specifically for the technique, or were too simple and thus performed badly in the original paper. Therefore, these are not considered.

Figure 3 shows the comparison of the three discussed techniques. Here, there is no doubt that the standard least amount of available colors performs best. It does look like both curves grow as fast for the number of vertices, but the growth

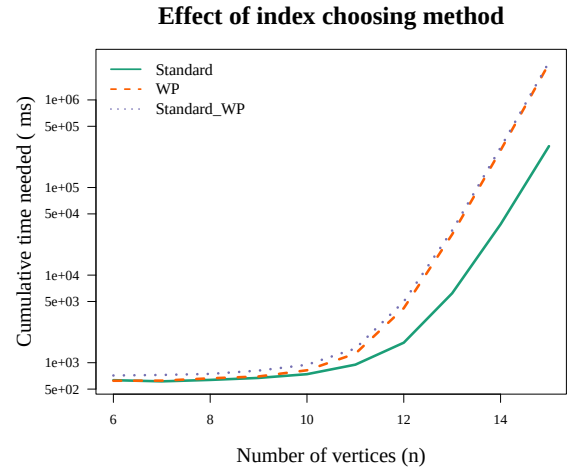


Figure 3: The choosing of next vertex indices is compared for 3 methods. WP and Standard_WP perform almost equal. The standard method grows the slowest.

phase of the standard technique starts later. Therefore, this index-choosing technique is used from here on out.

Second order optimization

Instead of only updating the available colors of vertices' neighbors, those of the neighbors' neighbors can also change when coloring a vertex. Here, this is only done when there is only one uncolored vertex left in the neighborhood of the neighbor. One of those rules is displayed in Algorithm 4 for the odd coloring. As mentioned before, the other colorings also have similar rules.

The rule for the unique-maximum coloring checks the number of times the maximum color occurs, if this is only once, prohibit the use of that color, otherwise, the uncolored vertex needs an even higher color to create a new maximum color.

The conflict-free rule checks whether any colors occur once, if not, the uncolored vertex should get a unique color. Otherwise, if there is only one color occurring once, this color should get avoided, if there are multiple, nothing happens.

The effect of using these second-order optimizations is visible in Figure 1. This optimization clearly makes the algorithm perform better. It even results in a slower growth of needed time.

Parallelization

A simple but effective optimization to apply is the use of parallelization. By using the already defined *RES/MOD* option in *plantri*, an option to split up the generated graphs into disjoint sets, the coloring of the graphs is easily done using multiprocessing. A procedure where a new process is created for each of the independent sets generated. This allows for a significant speedup dependent on the architecture of the system used (*i.e.*, *number of cores*). Because now more of the cores of the CPU are used, instead of only one. All outputs are then generated in different output files and combined at finish time.

Algorithm 5 `can_color_with_max_c_naive`

Parameters: The graph G , coloring K , max. color max_c and $index$

Output: Coloring achieved

```
1: if  $index \geq G.nb\_vertices$  then
2:   return is_correctly_colored( $G, K$ )
3: end if
4: for  $c$  from 1 to  $max\_c$  do
5:    $G.vertices[index].color \leftarrow c$ 
6:   if can_color_with_max_c_naive( $G, K, max\_c, index$ )
       then
7:     return true
8:   end if
9: end for
10: return false
```

A comparison between the normal and multiprocessed implementations is shown in Figure 1. As expected, for less graphs, splitting up all graphs, and combining the results will take longer than only coloring the graphs. After planar triangulations with 12 vertices, the speedup of using multiprocessing does become apparent.

Porting to C

The last optimization is the moving of the code from Java to C, a low-level programming language. The difference between the two languages is in the extra overhead found in Java. Java uses features like a garbage collector to monitor memory usage. C does not, which allows it to process graphs faster, but it does call for a more careful approach.

Only the best-performing datastructures and optimizations are added in C. These will therefore not be compared again in C.

Figure 1 shows a comparison between the Java second order implementation and the Final C implementation. The final version does have extra smaller optimizations, but even comparing both on almost the exact same implementation still favours C.

3.2 Correctness

The correctness of the implemented algorithm is never guaranteed. This is different for very simple versions of the algorithm. By comparing a naive implementation to the final implementation in C, discrepancies in coloring results can get found and resolved.

The naive algorithm is displayed in Algorithm 5. The code reflects the pseudo-code almost one-to-one. Therefore, it is clear that the naive implementation functions by testing every possible coloring given a maximum color. This implementation can hence be considered correct.

All planar triangulations with number of vertices $4 \leq n \leq 12$ were tested for all coloring variants. For the proper coloring, an extra tool called *nauty* developed by McKay and Piperno, is used [14]. This tool can find the chromatic numbers for any graph efficiently. Therefore, more planar triangulations were tested, those with number of vertices $4 \leq n \leq 20$. The improper colorings (e.g., iUMc) have lower chro-

Number of vertices	This program (s)	<i>Nauty</i> (s)	Ratio
10	0.013	0.003	4.3
11	0.014	0.004	3.5
12	0.020	0.008	2.5
13	0.064	0.039	1.64
14	0.39	0.25	1.56
15	2.9	1.9	1.54
16	22	15	1.49
17	177	121	1.46
18	1.4×10^3	1.0×10^3	1.41

Table 1: A comparison between the program developed in this paper and *nauty*. Compared for all planar triangulations with a given number of vertices. *Nauty* outperforms this program for all number of vertices. The ratio between the two tools goes down for more number of vertices.

matic numbers overall, therefore these were tested to a maximum of 14 vertices.

All tests found that the final implementation works correctly.

4 Results

Now that the researched problem and the developed algorithm have been discussed, the research questions are revisited.

The only coloring variant for which any widespread algorithms exist, is the proper coloring. As mentioned before, this includes tools like *nauty*. No open-source coloring tool for the other colorings exists. This means a comparison for efficiency for all coloring variants will not be possible.

Comparing *nauty*'s coloring option against the developed algorithm has expected results. As seen in Table 1, *nauty* is faster in coloring all planar triangulations with some number of vertices. It can be noted that the program developed here is still performing well. Around 1.4 times slower than *nauty*. For higher number of vertices, the ratio between the two is getting smaller, indicating the developed algorithm may become better for graphs with a high number of vertices.

Nauty is specifically designed for the coloring with the proper coloring. The other coloring variants discussed are not supported. These would also require the addition of completely new functions, and a lot of code duplication; or a complete rework of the algorithm. Therefore, *nauty* can not be easily expanded to new colorings, such as those discussed here, whereas this is possible for the developed algorithm. All coloring-dependent methods can be easily expanded without much added work. Additionally, *nauty* does not automatically support the use of multiprocessing, while this is supported for our implementation.

The other colorings are not comparable to other tools. Therefore, profiling the speed or efficiency of the algorithm is hard. But it does compare quite well to *nauty*, and is also very expandable, which does mean it is efficient in that sense.

The second research question concerned the use of the algorithm for disproving or substantiating conjectures. This is an exhaustive algorithm, therefore, all possible graphs for a given number of graphs can be checked to see if they follow

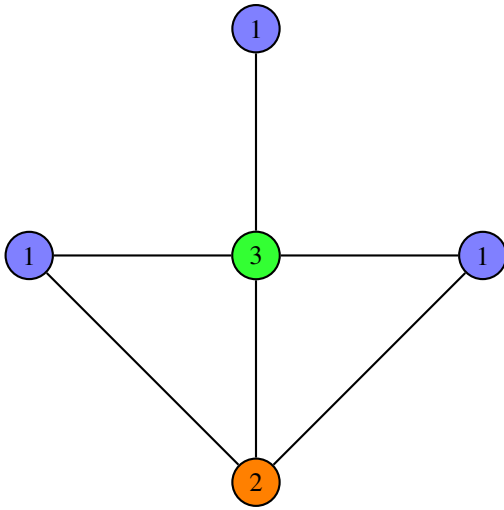


Figure 4: A colored graph for the pUMo coloring, where each vertex with color 1 can only have that color. Giving it any other color would result in an invalid coloring.

a conjecture or not.

Here, all planar triangulations—graphs with the highest possible number of edges and therefore graphs with a higher average chromatic number—with n vertices were checked ($4 \leq n \leq 20$). It was found that these graphs all conform to the current conjectures on chromatic numbers for all colorings.

The program can also assist in disproving the conjectures. An option is provided that will check all possible colorings of a given graph. This allows the user to assert if a graph has a certain property, helping in the search for gadgets—graphs which help in the disproving or proving of some theorem or conjecture.

The unique-maximum colorings have a property useful in the finding of gadgets, which the program can search for. As visible in Figure 4, there exist graphs with vertices that always have a certain color. These are great examples of gadgets, findable by the program. This is only applicable to the unique-maximum coloring variants, because as said before, permutations of other coloring variants still lead to a valid coloring. Therefore, for these other variants, no vertex can always have the same color.

5 Threats to validity

As explained in Section 3.2, the correctness of the implementation is never guaranteed. Furthermore, the comparison between implementations also doesn't guarantee correctness for graphs that were never tested, such as graphs with a high number of vertices. There is still a possibility that the naive implementation isn't implemented correctly. However, this is unlikely because of the perfect reflection between implementation and algorithm. Still, the `is_correctly_colored` check, seen in Algorithm 5, could function incorrectly.

The output of the proper coloring was tested against *nauty*. There is no guarantee of the correctness of *nauty*'s implementation.

All optimization experiments were performed on one specific machine, other computer architectures could yield a different outcome. Additionally, comparison of implementation decisions—the optimizations presented—are only tested at one implementation stage (*e.g.*, *naive implementation*, *post-bit-set implementation*), all tested in Java, and not at each stage of implementation. This could mean that some optimizations applied to the final program may result in different observations. For example, results of optimizations could differ when implemented in C.

6 Conclusion

In conclusion, this paper discussed a computational approach for the coloring of graphs. This is done for different variants of vertex colorings: namely the proper, odd, conflict-free and unique-maximum colorings.

A backtracking algorithm for coloring planar graphs with these coloring variants is discussed. Firstly, the algorithm was developed in Java, to then be ported over to C for efficiency. In Java, multiple optimizations were tested out and implemented. Different data structures were considered, a simple list performing the best. Additionally, other optimizations such as the usage of bit-sets or parallelization were also considered. All optimizations were tested to ensure the optimality of the algorithm.

The correctness of the algorithm was checked by comparing the outputs from the final program to those of a naive version. The simpleness of the naive algorithm and its structure showed the correctness, combined with identical outputs, lets us accept the final algorithm as correct.

This algorithm is the first of its kind, allowing the coloring of graphs for many different variants as one open-source program. It allows users to quickly color in graphs for many different variants, useful in applications such as wireless networks and more. The program isn't as fast as other coloring tools such as *nauty* for the proper coloring. But the expandability and easy internal usage of parallelization, things *nauty* lacks, partly make up for this drawback.

The conjectures surrounding the chromatic numbers for these colorings were examined further. It was found that the conjectures hold for all planar triangulations with less than or equal to 20 vertices. This gives some indication that these could hold for more graphs, since planar triangulations—in general—have a higher the chromatic number.

6.1 Future work

Future work could include the addition of more coloring variants into the program. This would allow the tool to be used for a wide range of problems. Additionally, other versions such as edge-, or face colorings could be included as well.

Next, the algorithm can be more optimized. For example multithreading when coloring one graph with a high number of vertices could speed up the program even further. Furthermore, other coloring rules, such as those described in Section 3.1, can be developed.

Lastly, further research could further improve bounds of chromatic numbers for the discussed coloring variants.

Use of genAI

For this paper, generative AI was used for the creation of figures. Additionally, it was also used in helping the writing process of the helper bash- and python files used in the implementation (e.g., *color.sh*).

References

- [1] Leonhard Euler. Solutio problematis ad geometriam situs pertinentis. *Commentarii academiae scientiarum Petropolitanae*, pages 128–140, 1741.
- [2] Kenneth Appel and Wolfgang Haken. Every planar map is four colorable. *Bulletin of the American Mathematical Society*, 82:711–712, 1976.
- [3] Mirko Petruševski and Riste Škrekovski. Colorings with neighborhood parity condition. *Discrete Applied Mathematics*, 321:385–391, 2022.
- [4] Guy Even, Zvi Lotker, Dana Ron, and Shakhar Smorodinsky. Conflict-free colorings of simple geometric regions with applications to frequency assignment in cellular networks. *SIAM Journal on Computing*, 33(1):94–136, 2003.
- [5] Shakhar Smorodinsky. Conflict-free coloring and its applications. In *Geometry—Intuitive, Discrete, and Convex: A Tribute to László Fejes Tóth*, pages 331–389. Springer, 2013.
- [6] Meir Katchalski, William McCuaig, and Suzanne Senger. Ordered colourings. *Discrete Mathematics*, 142(1):141–154, 1995.
- [7] Igor Fabrici, Borut Lužar, Simona Rindošová, and Roman Soták. Proper conflict-free and unique-maximum colorings of planar graphs with respect to neighborhoods. *Discrete Applied Mathematics*, 324:80–92, 2023.
- [8] Jan Petr and Julien Portier. The odd chromatic number of a planar graph is at most 8. *Graphs and Combinatorics*, 39(2):28, 2023.
- [9] Zachary Abel, Victor Alvarez, Erik D Demaine, Sándor P Fekete, Aman Gour, Adam Hesterberg, Phillip Keldenich, and Christian Scheffer. Conflict-free coloring of graphs. *SIAM Journal on Discrete Mathematics*, 32(4):2675–2702, 2018.
- [10] Fei Huang, Shanshan Guo, and Jinjiang Yuan. A short note on open-neighborhood conflict-free colorings of graphs. *SIAM Journal on Discrete Mathematics*, 34(3):2009–2015, 2020.
- [11] Lukáš Málík. Conflict-free colouring of planar graphs. Bakalářská práce, Masarykova univerzita, Fakulta informatiky Brno, 2023. Online; accessed 2025-11-26. Supervisor: Petr Hliněný.
- [12] Gunnar Brinkmann, Brendan McKay, et al. *Fast generation of planar graphs*. University of Kragujevac, 2007.
- [13] Murat Aslan and Nurdan Akhan Baykan. A performance comparison of graph coloring algorithms. *International Journal of Intelligent Systems and Applications in Engineering*, 4(Special Issue-1):1–7, 2016.
- [14] Brendan D McKay and Adolfo Piperno. Practical graph isomorphism, ii. *Journal of symbolic computation*, 60:94–112, 2014.